Engaging Howto Recipes Install Recipe MPI Rocky Linux VASP

Example Build of the VASP Software

About VASP

VASP is a first principles simulation tool for electronic structure and quantum mechanical molecular dynamics computations. The name VASP is an acronym of Vienna Ab-initio Simulation Package. The VASP software is used in quantum chemistry to simulate the properties and structure of atomic scale materials. VASP can compute detailed atomic structure of molecules, finding terms such as bond lengths and vibration frequencies.

Building VASP software

VASP is distributed as Fortran source code that must be compiled by end-users to create an executable program. This recipe describes how to compile VASP using the GNU compiler stack. The recipe shows commands for a Rocky Linux system.

Prerequisites

- To use VASP a research group must obtain a license from the VASP team as described here here.
- This example assumes you are working with a Rocky Linux environment.

1. Extract VASP source code files

Once a licensed copy of VASP has been obtained the source code files must be extracted from the tar file that can be downloaded by license holders from the VASP portal site. The command

```
tar -xzvf vasp.6.4.3.tgz
```

will extract the source files and their directory tree. This command should be executed in a sub-directory where you will store the compiled VASP programs.

Once the code has been extracted, switch to use the VASP directory for the remaining steps

```
cd vasp.6.4.3
```

Tip

For different versions of VASP, the download file name and directory name will be different. In that case, remember to adjust the example commands above accordingly.

2. Configure the compiler options file

The VASP software is distributed with multiple example compiler options files. These are in the sub-directory `arch/`. For this example we will use the GNU compiler options file `makefile.include.gnu_omp`. To activate the chosen options, copy the options file into the top-level VASP directory.

```
cp arch/makefile.include.gnu_omp makefile.include
```

3. Activate the relevant modules

To build the VASP program from the licensed source code several tools and libraries are needed. The modules below add the needed software. The `gcc` and `openmpi` modules provide compilers (gcc) and computational tools (openmpi) needed for parallel computing with VASP. The `lapack`, `scalapack`, `fftw` and `openblas` toos are numerical libraries that VASP uses.

```
module load gcc/12.2.0
module load openmpi/4.1.4
module load netlib-lapack/3.10.1
module load netlib-scalapack/2.2.0
module load fftw/3.3.10
module load openblas/0.3.26
```

4. Set environment variables that are needed for compilation

The compilation scripts that come with VASP include variables that must be set to the cluster's local values. Here we set environment variables to hold those settings.

```
SCALAPACK_ROOT=`module -t show  netlib-scalapack 2>&1 | grep CMAKE_PREFIX_PATH |
awk -F, '{print $2}'  | awk -F\" '{print $2}'`
FFTW_ROOT=`pkgconf --variable=prefix fftw3`
OPENBLAS_ROOT=$(dirname `pkgconf --variable=libdir openblas`)
```

5. Compile the VASP code

To compile the VASP code use the `make` program, passing it the environment variable settings as shown. The settings shown will also build the Fortran 90 modules that VASP includes.

```
make -j OPENBLAS_ROOT=$OPENBLAS_ROOT FFTW_ROOT=$FFTW_ROOT
SCALAPACK_ROOT=$SCALAPACK_ROOT MODS=1 DEPS=1
```

6. Check the VASP executables

The above commands should generate VASP executable programs `bin/vasp_std`, `bin/vasp_gam` and `bin/vasp_ncl`. To test that these programs can execute the following commands can be used:

```
export
LD_LIBRARY_PATH=${OPENBLAS_ROOT}/lib:${FFTW_ROOT}/lib:${SCALAPACK_ROOT}/lib
bin/vasp_std
```

If the code has compiled successfully the follow output should be generated. This output shows that the VASP program can be run. The output shows an error because no input files have been configured.

```
 -----------------------------------------------------------------------------
|                                                                             |
|     EEEEEEE  RRRRRR   RRRRRR   OOOOOOO  RRRRRR      ###     ###     ###     |
|     E        R     R  R     R  O     O  R     R     ###     ###     ###     |
|     E        R     R  R     R  O     O  R     R     ###     ###     ###     |
|     EEEEE    RRRRRR   RRRRRR   O     O  RRRRRR       #       #       #      |
|     E        R   R    R   R    O     O  R   R                                |
|     E        R    R   R    R   O     O  R    R      ###     ###     ###     |
|     EEEEEEE  R     R  R     R  OOOOOOO  R     R     ###     ###     ###     |
|                                                                             |
|     No INCAR found, STOPPING                                                |
|                                                                             |
|       ---->  I REFUSE TO CONTINUE WITH THIS SICK JOB ... BYE!!! <----       |
|                                                                             |
 -----------------------------------------------------------------------------

 -----------------------------------------------------------------------------
|                                                                             |
|     EEEEEEE  RRRRRR   RRRRRR   OOOOOOO  RRRRRR      ###     ###     ###     |
|     E        R     R  R     R  O     O  R     R     ###     ###     ###     |
|     E        R     R  R     R  O     O  R     R     ###     ###     ###     |
|     EEEEE    RRRRRR   RRRRRR   O     O  RRRRRR       #       #       #      |
|     E        R   R    R   R    O     O  R   R                                |
|     E        R    R   R    R   O     O  R    R      ###     ###     ###     |
|     EEEEEEE  R     R  R     R  OOOOOOO  R     R     ###     ###     ###     |
```

```
|                                                                             |
|     No INCAR found, STOPPING                                               |
|                                                                             |
|       ---->  I REFUSE TO CONTINUE WITH THIS SICK JOB ... BYE!!! <----       |
|                                                                             |
 -----------------------------------------------------------------------------

STOP 1
```

7. Example scripts to compile and test

The commands above can be combined into scripts as shown below. This example scripts that can either be run from the command line or submitted to Slurm as a batch job.

The following script shows compiling VASP and testing that the build completed successfully. Place and run this script from the directory where you put the VASP source .tgz file.

The call to `vasp_std` is expected to produce an error as in 6. Check the VASP executables above. To run a full VASP experiment problem specific inputs and parameters must be added to the script for running (see Running VASP below).

compile_and_test.sh

```
#!/bin/bash
#SBATCH --time=2:00:00
#SBATCH --partition=mit_normal
#SBATCH -c 8

tar -xzvf vasp.6.4.3.tgz
cd vasp.6.4.3
cp arch/makefile.include.gnu_omp makefile.include

module load gcc/12.2.0
module load openmpi/4.1.4
module load netlib-lapack/3.10.1
module load netlib-scalapack/2.2.0
module load fftw/3.3.10
module load openblas/0.3.26

SCALAPACK_ROOT=`module -t show  netlib-scalapack 2>&1 | grep CMAKE_PREFIX_PATH |
awk -F, '{print $2}'  | awk -F\" '{print $2}'`
FFTW_ROOT=`pkgconf --variable=prefix fftw3`
OPENBLAS_ROOT=$(dirname `pkgconf --variable=libdir openblas`)

make -j OPENBLAS_ROOT=$OPENBLAS_ROOT FFTW_ROOT=$FFTW_ROOT
SCALAPACK_ROOT=$SCALAPACK_ROOT MODS=1 DEPS=1

export
LD_LIBRARY_PATH=${OPENBLAS_ROOT}/lib:${FFTW_ROOT}/lib:${SCALAPACK_ROOT}/lib
```

```
srun ./bin/vasp_std
```

Creating a VASP Module

It can be convenient to create a module for VASP since it does have several dependencies. Below is an example modulefile. This modulefile assumes you have installed VASP 6.4.3, placed it in `$HOME/software/VASP`, and used the same dependency modules to build VASP as described in Step 3 above. If you have installed a different version of VASP, placed it in a different location, or used different dependency modules you will need to adjust the modulefile accordingly.

$HOME/software/modulefiles/vasp/6.4.3.lua

```
-- -*- lua -*-
--

whatis([[Name : vasp]])
whatis([[Version : 6.4.3]])
whatis([[Target : x86_64]])
whatis([[Short description : The Vienna Ab initio Simulation Package (VASP) is a com
mechanical molecular dynamics, from first principles.]])

local base = pathJoin(os.getenv("HOME"),"software/VASP/vasp.6.4.3")

depends_on("gcc/12.2.0")
depends_on("openmpi/4.1.4")

prepend_path("LD_LIBRARY_PATH","/orcd/software/core/001/spack/pkg/openblas/0.3.26/ro
scalapack/2.2.0/ff5iskg/./lib")
prepend_path("PATH", pathJoin(base,"bin/"), ":")
```

Running VASP

To run VASP create a job script like the one below in the same directory as your input files. You may need to increase `ntasks` or `cpus-per-task` or allocation additional resources depending on the size of the problem. This script assumes you have created a module and placed the modulefile in in `$HOME/software/modulefiles`. Update the location of your VASP module as needed. VASP has a page of examples in their documentation that can be used for testing.

run_vasp.sh

```
#!/bin/bash
#SBATCH --time=2:00:00
```

```
#SBATCH --partition=mit_normal
#SBATCH --ntasks=4 # Number of VASP processes
#SBATCH --cpus-per=task=2 # Number of threads per VASP process

# Limit the number of threads to the number of cpus requested
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK

# Load the VASP module
module use $HOME/software/modulefiles
module load vasp/6.4.3

# Run VASP
srun vasp_std
```

Note

During testing we found that VASP has a tendency to create a very large number of threads that can slow down the calculations and cause them to hang. To prevent that we've set the `$OMP_NUM_THREADS` environment variable to the number of `cpus-per-task` requested (`$SLURM_CPUS_PER_TASK`).

August 1, 2025